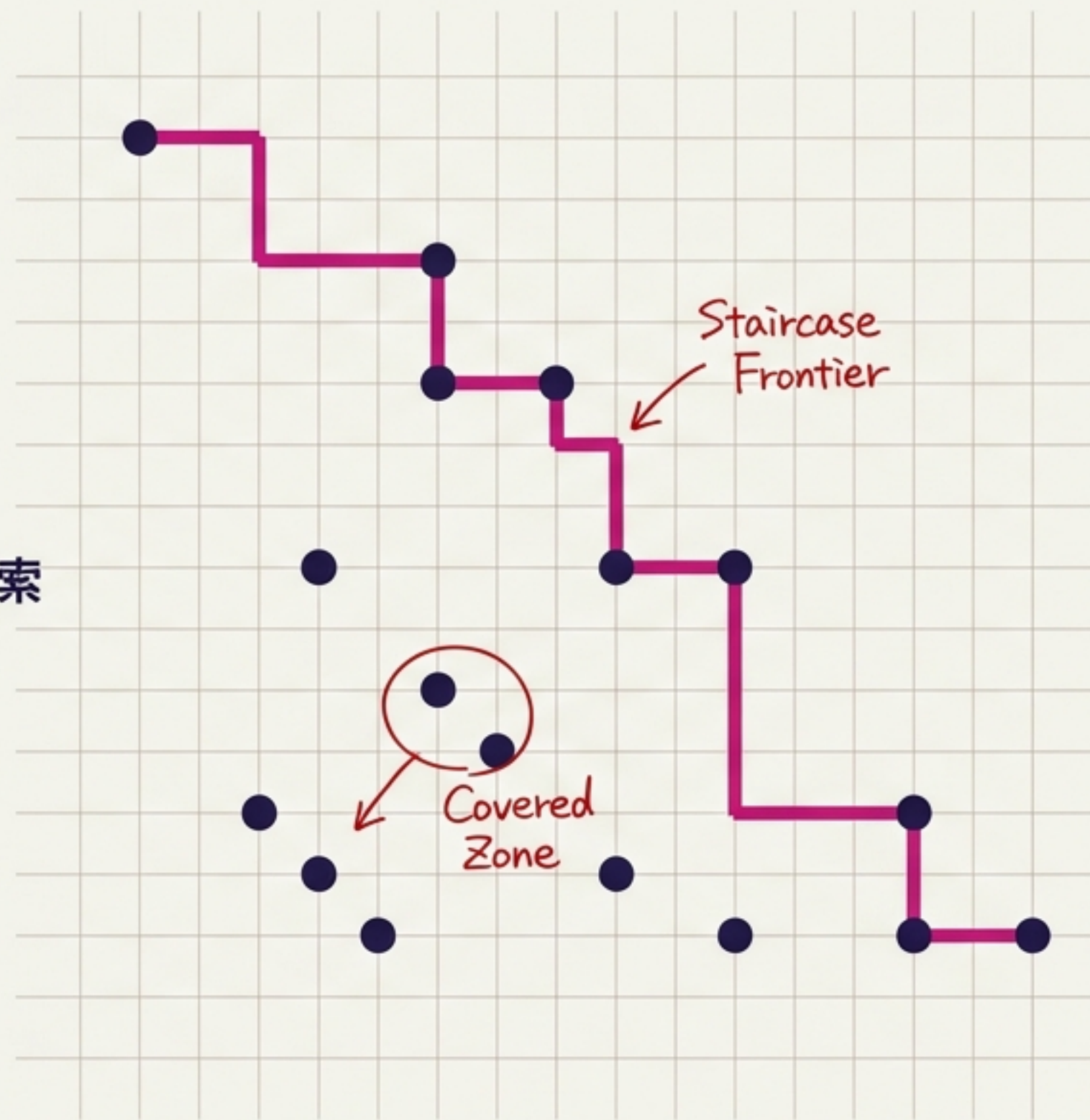


AtCoder ABC462 C Not Covered Points

競技プログラミング解説：2次元平面上的フロンティア探索

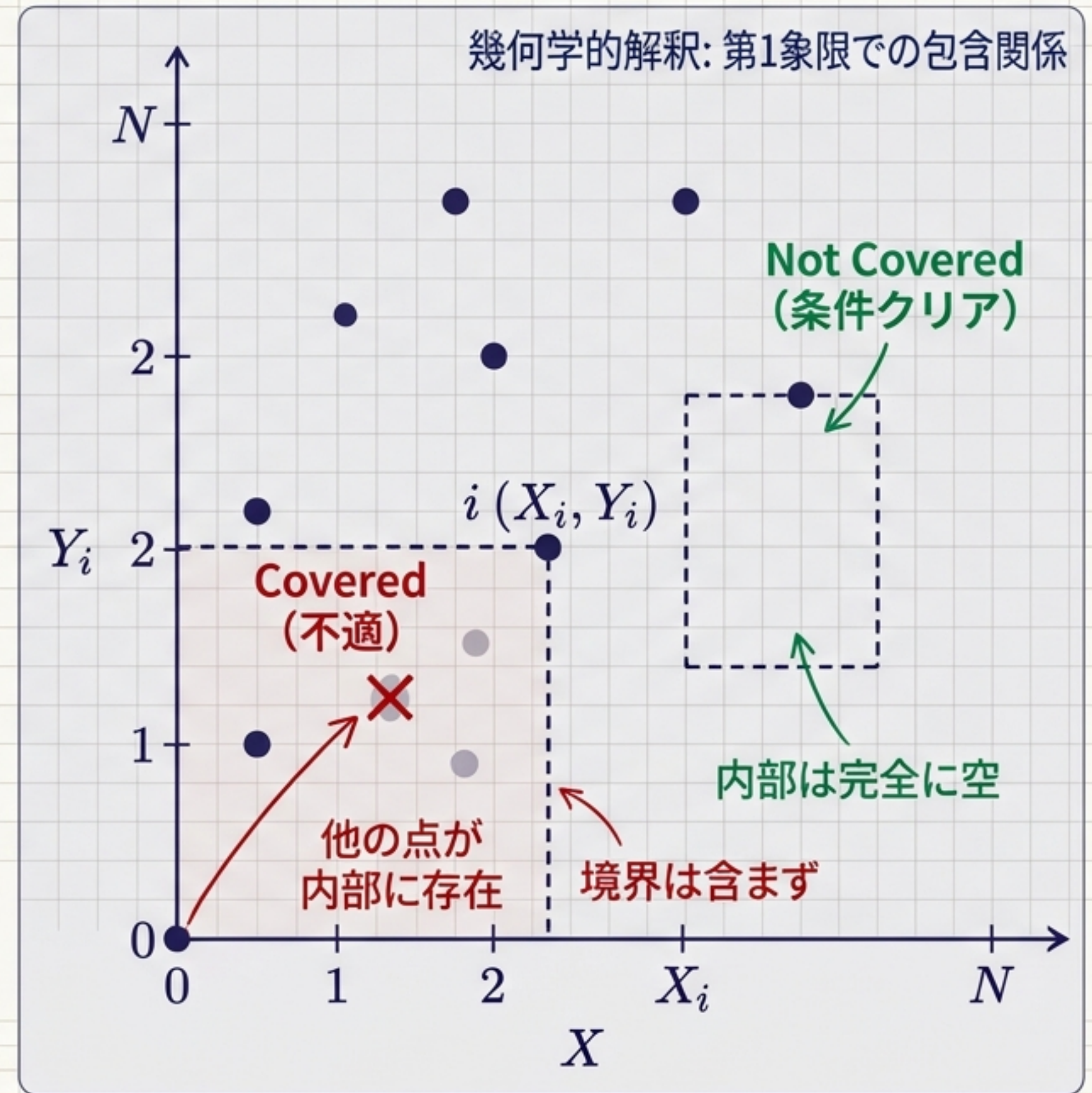


CodeQUEEN 2026 -qual-
難易度：300点
目標計算量： $O(N \log N)$

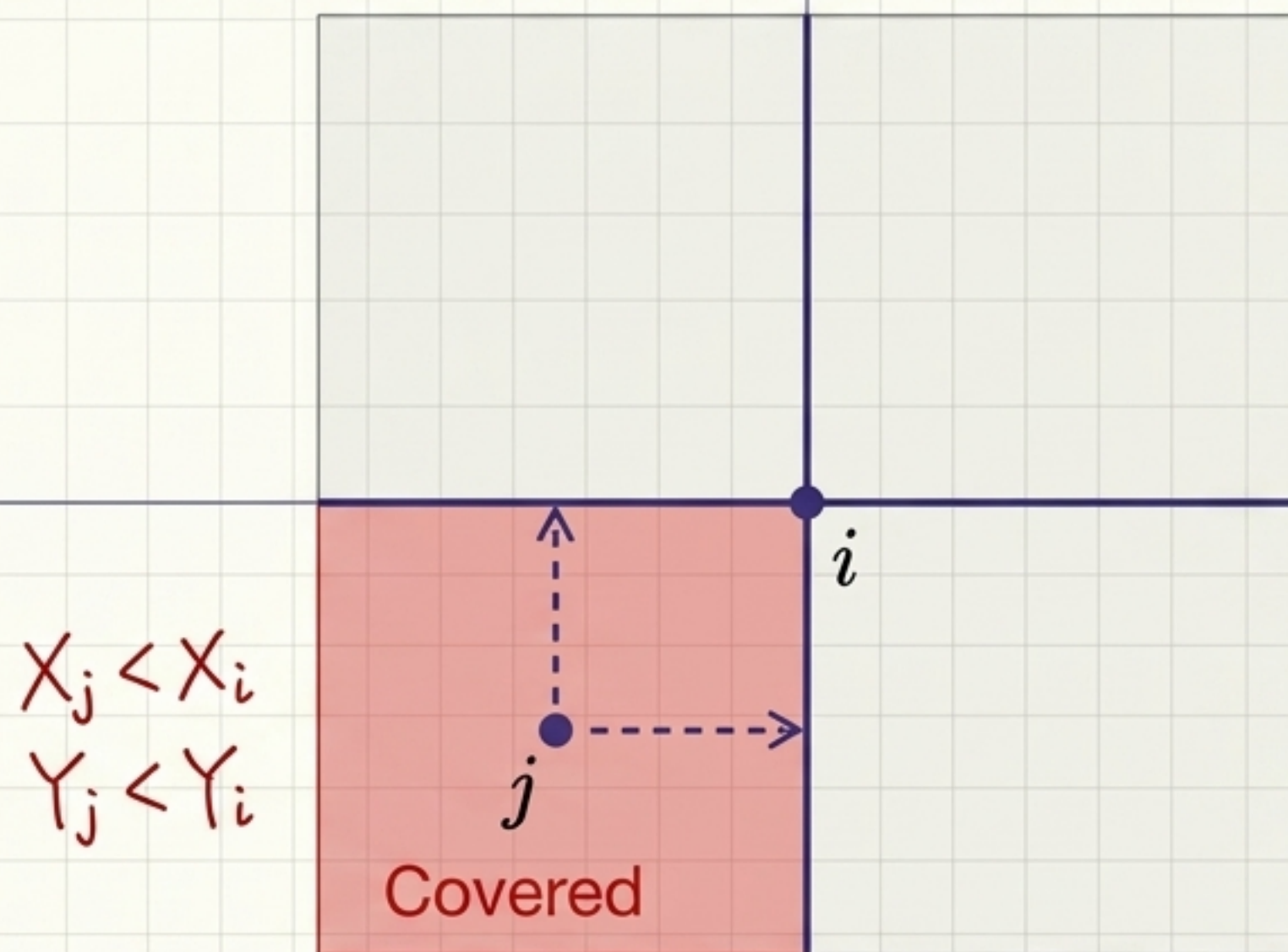
軸平行な長方形と「Not Covered」の定義

- 点: 2次元平面上に N 個の点 ($1 \leq N \leq 3 \times 10^5$)
- 制約: X, Y 座標はそれぞれ 1 から N までの順列 (同じ座標の重複なし)
- 条件: 左下 $(0, 0)$ 、右上 (X_i, Y_i) を結ぶ長方形の内部に、他の点が1つも存在しない点 i の個数を数える (境界線は含まない)

※ ここでの「Covered」は、点 i が定義する長方形内部に他の点が存在することを指す。



幾何学的な言い換え：真の左下

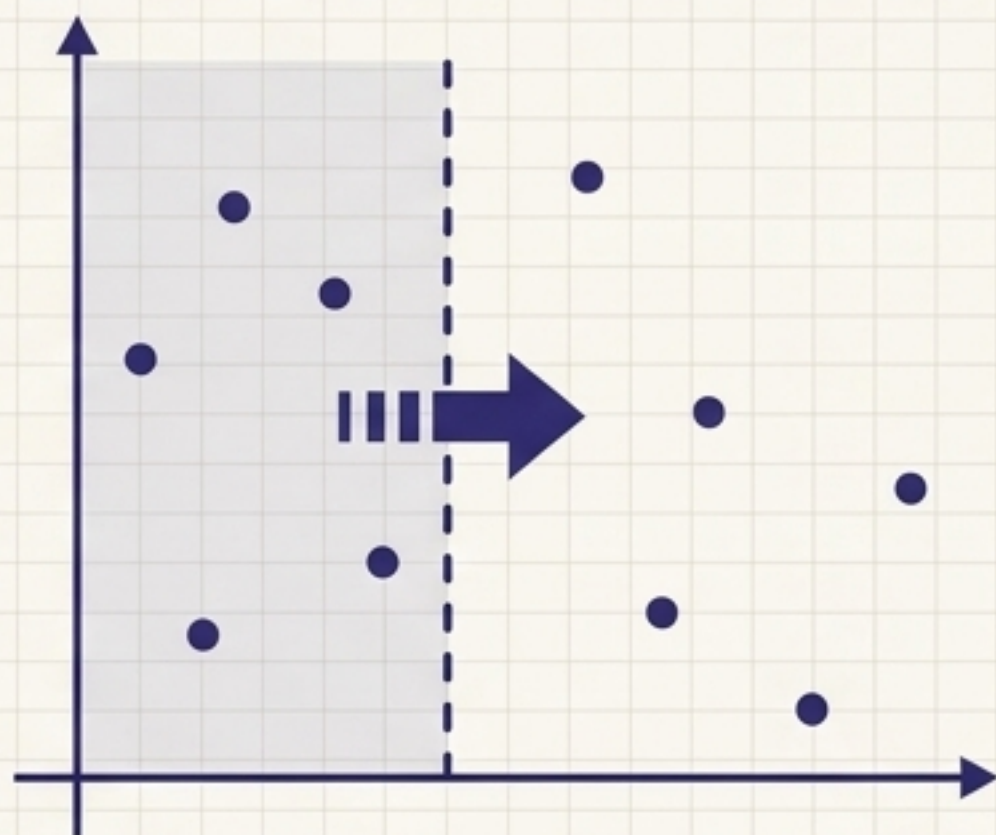


点 j が点 i の長方形の内部に入る条件 $\Leftrightarrow X_j < X_i$ かつ $Y_j < Y_i$

結論: 長方形を意識する必要はない。「自分の左下に点が1つも存在しない」点を探す問題に帰着。

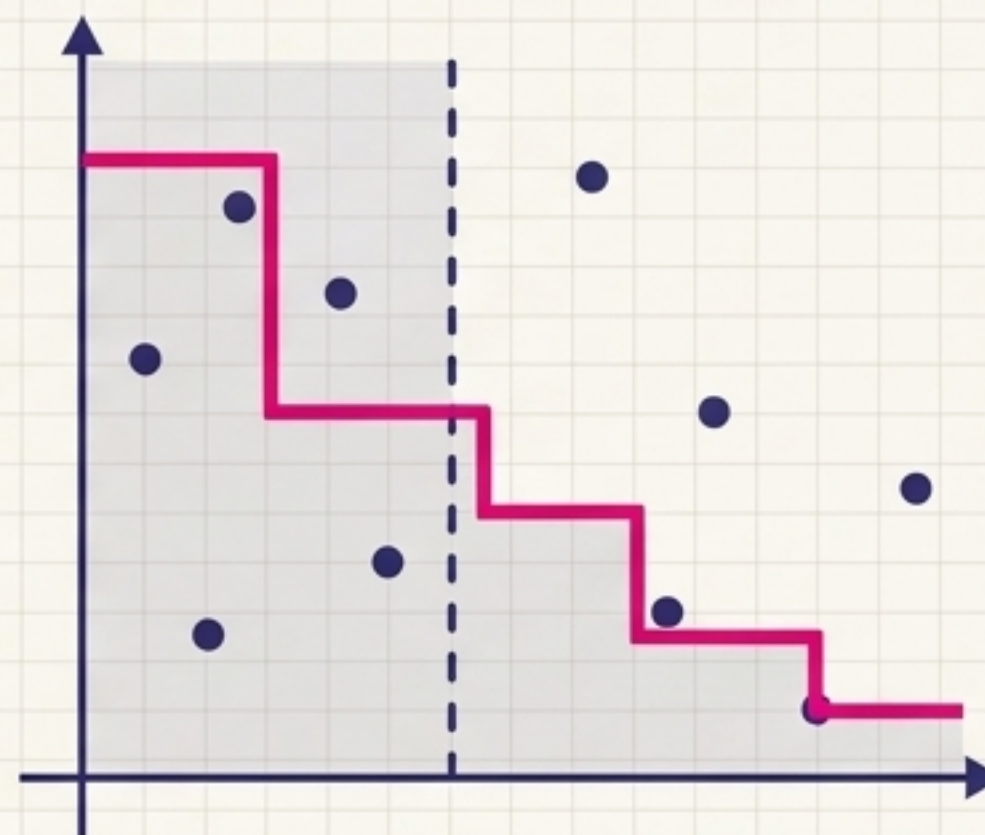
最適化アプローチ：ソートとPrefix Minimum

Step 1: X軸の走査 (X昇順ソート)



Xの昇順に走査することで、「自分より左 (Xが小さい)」点はすべて確認済みの状態を作る。

Step 2: Y軸のフロンティア管理



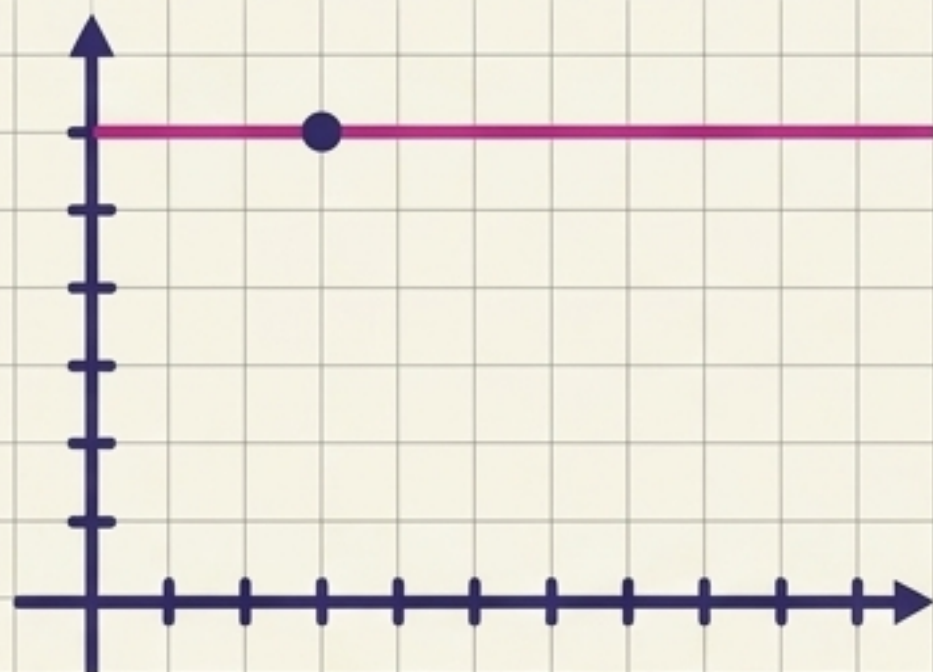
過去に見たYの最小値 minY さえ記憶しておけば判定可能。

- $Y_i > \text{minY} \rightarrow$ 左下に点あり (Covered)
- $Y_i < \text{minY} \rightarrow$ 左下に点なし (Not Covered: $\text{ans}++$, minY 更新)

↙ 最小値更新!

サンプルトレース：N=3 の挙動

入力: (2,1), (1,3), (3,2) → X昇順ソート: (1,3), (2,1), (3,2)

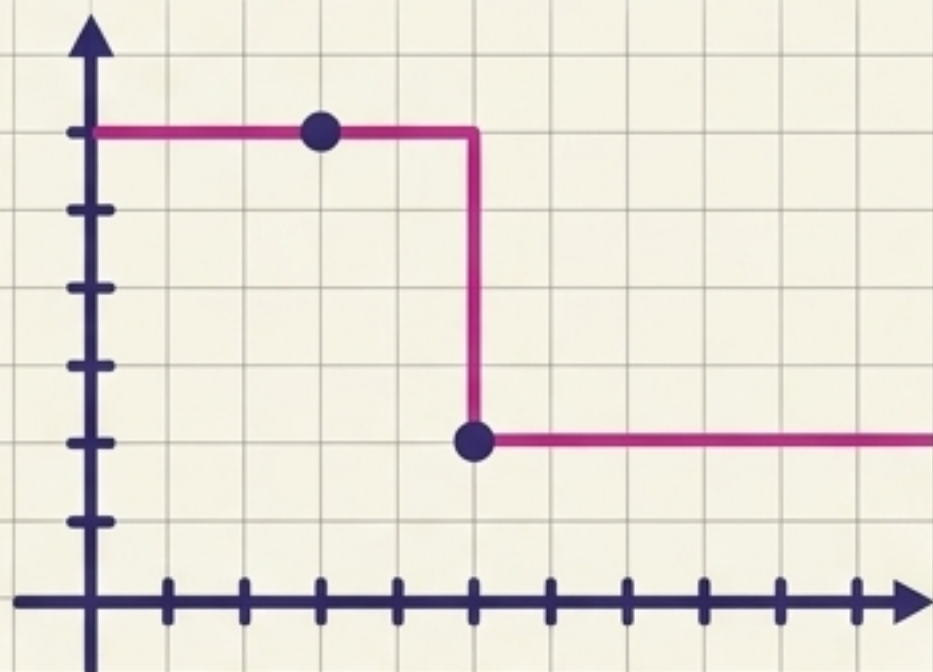


Step 1: 点 (1, 3)

直前 $\text{minY} = \infty$

$3 < \infty$ (Yes) → Not Covered ✓

ans = 1, minY = 3 に更新

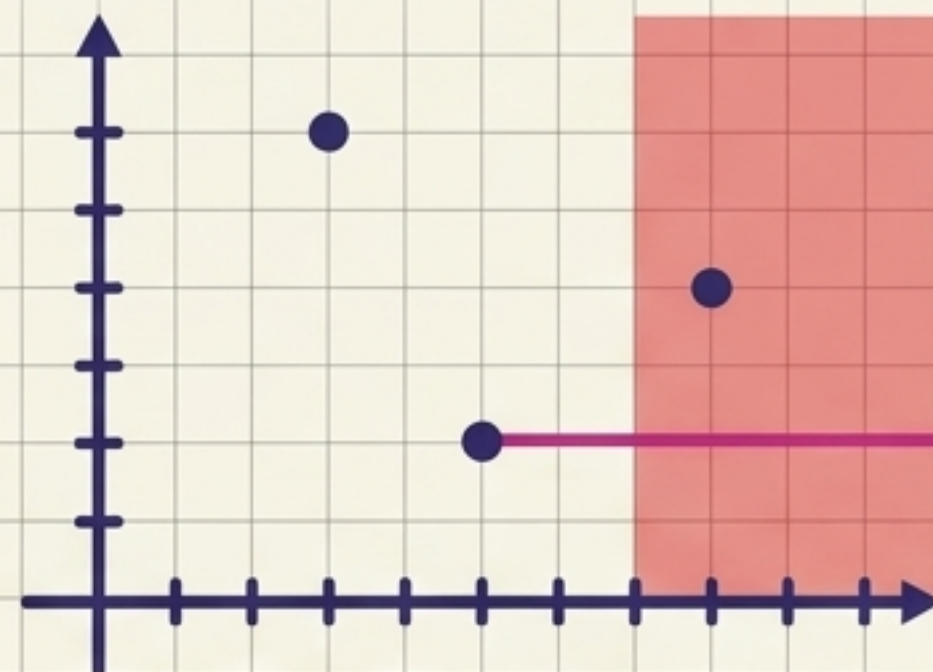


Step 2: 点 (2, 1)

直前 $\text{minY} = 3$

$1 < 3$ (Yes) → Not Covered ✓

ans = 2, minY = 1 に更新



Step 3: 点 (3, 2)

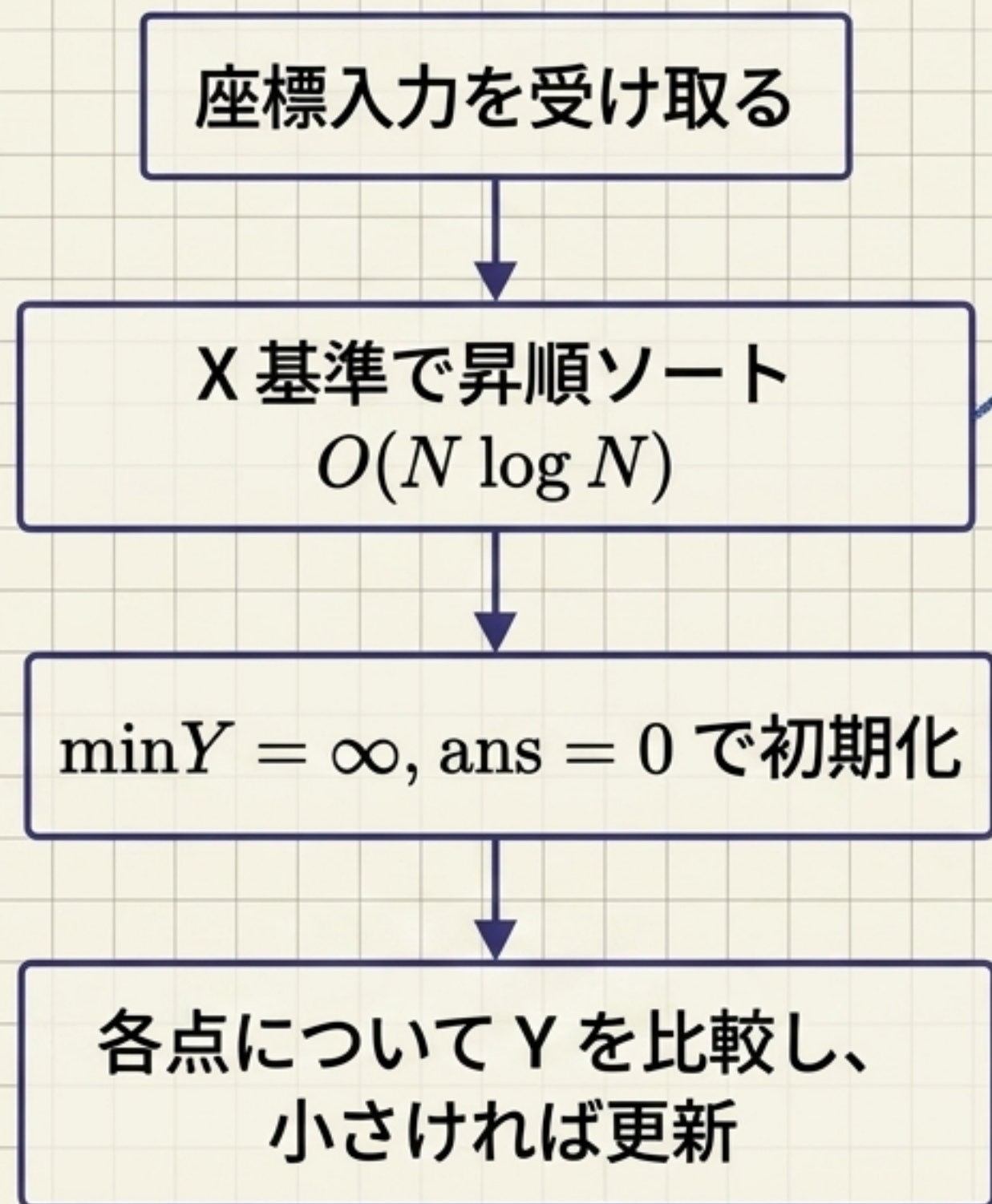
直前 $\text{minY} = 1$

$2 < 1$ (No) → Covered ✗

ans = 2, minY 変化なし

最終的な答え: 2

アルゴリズムの実装 (Python)



```
points.sort(key=lambda p: p[0])
```

```
min_y = float('inf')
```

```
ans = 0
```

```
for x, y in points:
```

```
    if y < min_y:
```

```
        ans += 1
```

```
        min_y = y
```

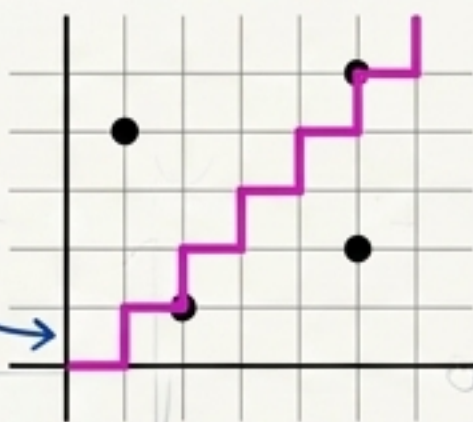
計算量分析と「双対」のアプローチ

時間計算量： $O(N \log N)$ — ソート処理が支配的。全探索 $O(N^2)$ の TLE を完全に回避し、1パス $O(N)$ の走査で完結。

左下フロンティア

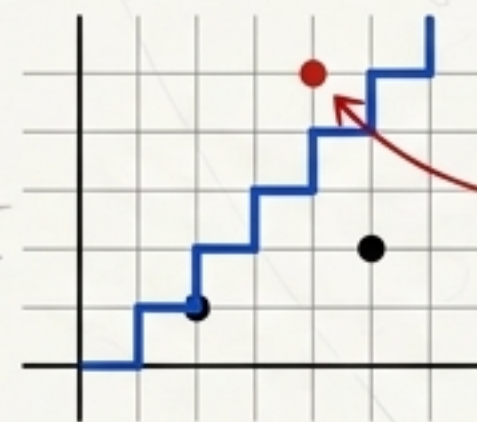
- $X_j < X_i$ かつ $Y_j < Y_i$
- 左下から迫る階段状の境界線

現在地点
(X_i, Y_i)の左下



右上フロンティア

- $X_i < X_j$ かつ $Y_i < Y_j$
- 自分より右上に点があるか



自分より右上の
存在確認

結論: どちらの視点で思考しても、最終的な本質は「ソートして片側の最小/最大を累積値で持つ」という階段（フロンティア）の構造に帰着する。

※典型参照：ABC091 C 「2D Plane 2N Points」と同系統のアプローチ